| | |
|---|---|
| **Started on** | Tuesday, 31 March 2026, 9:35 AM |
| **State** | Finished |
| **Completed on** | Tuesday, 31 March 2026, 9:36 AM |
| **Time taken** | 35 secs |
| **Grade** | **0.00** out of 100.00 |

**Question 1**

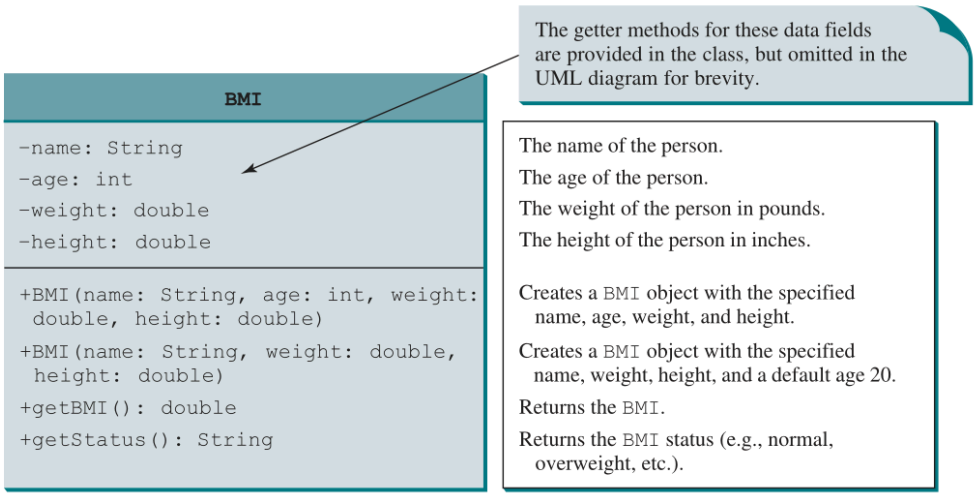
Not answered

Mark 0.00 out of 25.00

We have previously presented a program for computing the body mass index (BMI). The code cannot be reused in other programs, because the code is in the main method. To make it reusable, define a static method to compute body mass index as follows:

**public static double** getBMI(**double** weight, **double** height)

Consider the following UML diagram to develope a class for computing BMI.




**Answer:** (penalty regime: 0, 0, 0, 100 %)

```
1
```

**Show/hide question author's solution (Java)**

```java
public class BMI {
 private String name;
 private int age;
 private double weight; // in pounds
 private double height; // in inches
 public static final double KILOGRAMS_PER_POUND = 0.45359237;
 public static final double METERS_PER_INCH = 0.0254;

 public BMI(String name, int age, double weight, double height) {
 this.name = name;
 this.age = age;
 this.weight = weight;
 this.height = height;
 }

 public BMI(String name, double weight, double height) {
```

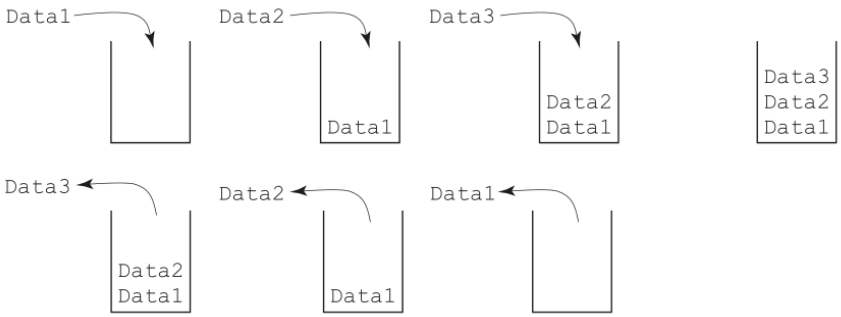
```
16  public BMI(String name, double weight, double height) {
17  this(name, 20, weight, height);
18  }
19
20  public double getBMI() {
21  double bmi = weight * KILOGRAMS_PER_POUND /
22  ((height * METERS_PER_INCH) * (height * METERS_PER_INCH));
23  return Math.round(bmi * 100) / 100.0;
24  }
25
26  public String getStatus() {
27  double bmi = getBMI();
28  if (bmi < 18.5)
29  return "Underweight";
30  else if (bmi < 25)
31  return "Normal";
32  else if (bmi < 30)
33  return "Overweight";
34  else
35  return "Obese";
36  }
37
38  public String getName() {
39  return name;
40  }
41
42  public int getAge() {
43  return age;
44  }
45
46  public double getWeight() {
47  return weight;
48  }
49
50  public double getHeight() {
51  return height;
52  }
53 }
54
55 /*
56 public class UseBMIClass {
57  public static void main(String[] args) {
58  BMI bmi1 = new BMI("Kim Yang", 18, 145, 70);
59  System.out.println("The BMI for " + bmi1.getName() + " is "
60  + bmi1.getBMI() + " " + bmi1.getStatus());
61
62  BMI bmi2 = new BMI("Susan King", 215, 70);
63  System.out.println("The BMI for " + bmi2.getName() + " is "
64  + bmi2.getBMI() + " " + bmi2.getStatus());
65  }
66 }
67 */
```

**Question 2**

Not answered

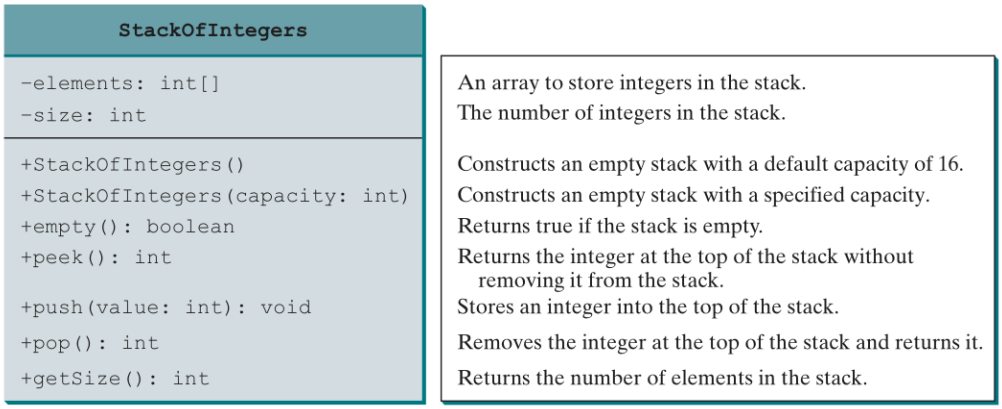
Mark 0.00 out of 25.00

Recall that a stack is a data structure that holds data in a last-in, first-out fashion, as shown in below.




Suppose you wish to define a class to model stacks. For simplicity, assume the stack holds the **int** values.

Thus, name the stack class **StackOfIntegers**. The UML diagram for the class is shown in the following figure.




**Answer:** (penalty regime: 0, 0, 0, 100 %)

```
1
```

**Show/hide question author's solution (Java)**

```
/*public class TestStackOfIntegers {
 public static void main(String[] args) {
 StackOfIntegers stack = new StackOfIntegers();
for (int i = 0; i < 10; i++)
 stack.push(i);

 while (!stack.empty())
 System.out.print(stack.pop() + " ");
 }
```

```
}*/

public class StackOfIntegers {
 private int[] elements;
 private int size;
 public static final int DEFAULT_CAPACITY = 16;

 /** Construct a stack with the default capacity 16 */
 public StackOfIntegers() {
 this(DEFAULT_CAPACITY);
 }

 /** Construct a stack with the specified capacity */
 public StackOfIntegers(int capacity) {
 elements = new int[capacity];
 }
/** Push a new integer to the top of the stack */
 public void push(int value) {
 if (size >= elements.length) {
 int[] temp = new int[elements.length * 2];
 System.arraycopy(elements, 0, temp, 0, elements.length);
 elements = temp;
 }

 elements[size++] = value;
 }

 /** Return and remove the top element from the stack */
 public int pop() {
 return elements[--size];
 }

 /** Return the top element from the stack */
 public int peek() {
 return elements[size - 1];
 }

 /** Test whether the stack is empty */
 public boolean empty() {
 return size == 0;
 }

 /** Return the number of elements in the stack */
 public int getSize() {
 return size;
 }
}
```

**Question 3**

Not answered

Mark 0.00 out of 25.00

(The Time class) Design a class named Time. The class contains:

■ The data fields hour, minute, and second that represent a time.

■ A no-arg constructor that creates a Time object for the current time. (The values of the data fields will represent the current time.)

A constructor that constructs a Time object with a specified elapsed time since midnight, January 1, 1970, in milliseconds. (The values of the data

fields will represent this time.)

■ A constructor that constructs a Time object with the specified hour, minute, and second.

■ Three getter methods for the data fields hour, minute, and second, respectively.

■ A method named setTime(long elapseTime) that sets a new time for the object using the elapsed time. For example, if the elapsed time is 555550000 milliseconds, the hour is 10, the minute is 19, and the second is 10. Draw the UML diagram for the class then implement the class.

Write a test program that creates three Time objects (using new Time(), new Time(555550000), and new Time(5, 23, 55)) and displays their hour, minute, and second in the format **hour:minute:second**.

(Hint: The first two constructors will extract the hour, minute, and second from the elapsed time. For the no-arg constructor, the current time can be obtained using System.currentTimeMillis(), as shown in Listing 2.7, Show CurrentTime.java. Assume the time is in GMT.)

**Answer:** (penalty regime: 0, 0, 0, 100 %)

```
1
```

▼ **Show/hide question author's solution (Java)**

```java
/*public class Exercise10_01 {
  public static void main(String[] args) {
    MyTime time1 = new MyTime();
    System.out.println(time1.getHour() + ":" +
      time1.getMinute() + ":" + time1.getSecond());

    MyTime time2 = new MyTime(555550000);
    System.out.println(time2.getHour() + ":" +
      time2.getMinute() + ":" + time2.getSecond());

    MyTime time3 = new MyTime(5, 23, 55);
    System.out.println(time3.getHour() + ":" +
      time3.getMinute() + ":" + time3.getSecond());
  }
}
*/

class MyTime {
  private int hour;
  private int minute;
  private int second;
```

```
22
23    public MyTime() {
24      this(System.currentTimeMillis());
25    }
26
27    public MyTime(long elapsedTime) {
28      setTime(elapsedTime);
29    }
30
31    public MyTime(int hour, int minute, int second) {
32      this.hour = hour;
33      this.minute = minute;
34      this.second = second;
35    }
36
37    public int getHour() {
38      return hour;
39    }
40
41    public int getMinute() {
42      return minute;
43    }
44
45    public int getSecond() {
46      return second;
47    }
48
49    public void setTime(long elapsedTime) {
50      // Obtain the total seconds since the midnight, Jan 1, 1970
51      long totalSeconds = elapsedTime / 1000;
52
53      // Compute the current second in the minute in the hour
54      second = (int)(totalSeconds % 60);
55
56      // Obtain the total minutes
57      long totalMinutes = totalSeconds / 60;
58
59      // Compute the current minute in the hour
60      minute = (int)(totalMinutes % 60);
61
62      // Obtain the total hours
63      int totalHours = (int)(totalMinutes / 60);
64
65      // Compute the current hour
66      hour = (int)(totalHours % 24);
67    }
68  }
69
```

**Question 4**

Not answered

Mark 0.00 out of 25.00

(The MyInteger class) Design a class named MyInteger. The class contains:

■ An int data field named value that stores the int value represented by this object.

■ A constructor that creates a MyInteger object for the specified int value.

■ A getter method that returns the int value.

■ The methods isEven(), isOdd(), and isPrime() that return true if the value in this object is even, odd, or prime, respectively.

■ The static methods isEven(int), isOdd(int), and isPrime(int) that return true if the specified value is even, odd, or prime, respectively.

■ The static methods isEven(MyInteger), isOdd(MyInteger), and isPrime(MyInteger) that return true if the specified value is even, odd, or prime, respectively.

■ The methods equals(int) and equals(MyInteger) that return true if the value in this object is equal to the specified value.

■ A static method parseInt(char[]) that converts an array of numeric characters to an int value.

■ A static method parseInt(String) that converts a string into an int value.

Write a client program that tests all methods in the class.

**Answer:** (penalty regime: 0, 0, 0, 100 %)

```
1
```

▼ **Show/hide question author's solution (Java)**

```java
/*public class Exercise10_03 {
  public static void main(String[] args) {
    MyInteger n1 = new MyInteger(5);
    System.out.println("n1 is even? " + n1.isEven());
    System.out.println("n1 is prime? " + n1.isPrime());
    System.out.println("15 is prime? " + MyInteger.isPrime(15));

    char[] chars = {'3', '5', '3', '9'};
    System.out.println(MyInteger.parseInt(chars));

    String s = "3539";
    System.out.println(MyInteger.parseInt(s));

    MyInteger n2 = new MyInteger(24);
    System.out.println("n2 is odd? " + n2.isOdd());
    System.out.println("45 is odd? " + MyInteger.isOdd(45));
    System.out.println("n1 is equal to n2? " + n1.equals(n2));
    System.out.println("n1 is equal to 5? " + n1.equals(5));
  }
}*/

class MyInteger {
  private int value;

```

```
25  public int getValue() {
26    return value;
27  }
28
29  public MyInteger(int value) {
30    this.value = value;
31  }
32
33  public boolean isPrime() {
34    return isPrime(value);
35  }
36
37  public static boolean isPrime(int num) {
38    if ((num == 1) || (num == 2)) {
39      return true;
40    }
41
42    for (int i = 2; i <= num / 2; i++) {
43      if (num % i == 0) {
44        return false;
45      }
46    }
47
48    return true;
49  }
50
51  public static boolean isPrime(MyInteger o) {
52    return isPrime(o.getValue());
53  }
54
55  public boolean isEven() {
56    return isEven(value);
57  }
58
59  public boolean isOdd() {
60    return isOdd(value);
61  }
62
63  public static boolean isEven(int n) {
64    return n % 2 == 0;
65  }
66
67  public static boolean isOdd(int n) {
68    return n % 2 != 0;
69  }
70
71  public static boolean isEven(MyInteger n) {
72    return isEven(n.getValue());
73  }
74
75  public boolean equals(int anotherNum) {
76    return value == anotherNum;
77  }
78
79  public boolean equals(MyInteger o) {
80    return value == o.getValue();
81  }
82
83  public static int parseInt(char[] numbers) {
84    // numbers consists of digit characters.
85    // For example, if numbers is {'1', '2', '5'}, the return value
86    //  should be 125. Please note that
87    // numbers[0] is '1'
88    // numbers[1] is '2'
89    // numbers[2] is '5'
90
91    int result = 0;
92    for (int i = 0; i < numbers.length; i++) {
93      result = result * 10 + (numbers[i] - '0');
94    }
95
96    return result;
97  }
98
99  // You may mention this when you covered Ch8
100 public static int parseInt(String s) {
101   // s consists of digit characters.
102   // For example, if s is "125", the return value
103   //  should be 125.
104   int result = 0;
105   for (int i = 0; i < s.length(); i++) {
```

```
105     for (int i = 0; i < s.length(); i++) {
106       result = result * 10 + (s.charAt(i) - '0');
107     }
108
109     return result;
110   }
111 }
112
```